

Introduction

1. Domain-Specific area

Online reviews have become a central component in decision-making for modern consumers, particularly in sectors such as retail, hospitality, and food services. These reviews act as trust signals when consumers are unable to assess products or services physically. However, the rise of fake reviews, fabricated endorsements, or criticisms created for financial or reputational gain poses a serious threat to both consumers and honest businesses. Research shows that up to 30% of online reviews may be deceptive, especially in high-stakes industries such as hospitality, electronics, beauty products (Jindal & Liu, 2008). A recent study published in the *Journal of Business Research* highlights that exposure to fake reviews significantly deteriorates customers' perception of brand credibility. Furthermore, businesses engaging in deceptive review practices risk severe reputational damage, regulatory penalties, and long-term erosion of consumer trust. Fake reviews can distort public opinion, manipulate online ratings, and mislead potential customers. As Channel News reported, popular eateries in Singapore, including Ishiro and Mentai-Ya, were caught using misleading online reviews to gain unfair visibility, drawing criticism from the public and regulatory scrutiny (Channel News Asia, 2021). Similarly, Todd Cue (2021) highlights that restaurants in the UK face tangible revenue losses when targeted by fake negative reviews, as even a few misleading ratings can drastically alter customer perception. Given this context, the task of automatically classifying online reviews as either authentic or fabricated has become both relevant and impactful. Text classification, binary classification can be used to identify fake reviews using supervised machine learning. By comparing both traditional statistical models and deep learning models, this project aims to evaluate the most effective methodology for accurately classifying fake content. This domain presents a unique opportunity to assess the performance of statistical models like Naive Bayes or logistic regression, etc.

2. Objectives

The objective of this project is to evaluate and compare the performance of traditional statistical and modern embedding-based models in the context of fake review detection, a pressing issue in digital commerce. As e-commerce platforms, food delivery services, and review aggregators increasingly influence consumer behaviour, the ability to reliably differentiate between genuine and deceptive content becomes crucial for maintaining trust and fairness. This project aims to address this issue by developing classification models capable of distinguishing between real and fake reviews with high accuracy.

1. Applying both statistical models, such as Logistic Regression, Naive Bayes using Bag of Words or TF-IDF, and embedding-based deep learning, such as LSTM using tokenised word sequences.

2. Preprocessing text data through lemmatization, stopword removal, and vectorization to ensure optimal input representation.
3. Evaluating the strengths and limitations of traditional vs deep learning methods in handling noisy, real-world data.
4. Identifying which model is better suited for distinguishing between genuine and fake reviews in terms of accuracy, generalizability, and interpretability.

This comparison is significant because fake reviews are often written to resemble authentic language. Hence, understanding which approach, statistical frequency-based models or context-sensitive embeddings, performs better in this scenario can provide insights into designing more robust detection systems. Overall, the findings could be used to inform review moderation systems on e-commerce platforms, which would lead to improving consumer trust and platform integrity.

3. Dataset Objectives

The dataset used in this project is the Fake Reviews Dataset published by Maxwell on Kaggle. It was specifically constructed for training models to distinguish between computer-generated (CG) fake reviews and original(OR) human-written reviews. This binary classification challenge aligns with the objectives and provides a structured foundation for exploring both statistical and embedding-based machine learning methods. The dataset contains:

40,000+ labelled reviews in total Two distinct classes: CG(Computer-generated): Reviews produced using AI or templated text to simulate fake customer feedback. OR(original reviews): Authentic reviews written by real customers. A primary text column labelled `cleaned_text`, which has been preprocessed for whitespace and punctuation. A label column indicating the review class as 'CG' or 'OR'. This dataset was built by mixing genuine reviews collected from real platforms with synthetically generated ones created using text generation techniques. This ensures that the fake reviews simulate realistic, deceptive language patterns while also posing a non-trivial classification problem. The fake reviews were generated by the early OpenAI's GPT model, one of the early releases, and the real reviews were extracted from authentic Amazon product review data, which ensures organic user-generated feedback across a range of items. This dataset is suitable due to its binary classification format, which is ideal for comparing statistical and embedding-based models. It is a balanced sample size, which reduces bias and simplifies evaluation using standard metrics like accuracy, precision, and F1-score.

4. Evaluation Methodology

The models are evaluated based on four widely used classification metrics. Accuracy: The overall correctness of the model's prediction Precision: The proportion of true positives among all positive predictions. Recall: The proportion of true positives identified among all

actual positives. F1-Score: The harmonic mean of precision and recall. These metrics are particularly important in imbalanced classification tasks, where relying solely on accuracy could be misleading. For example, if fake reviews are a minority class, a model that always predicts 'original' could still show high accuracy while failing to detect any fake reviews. The metrics are computed using scikit-learn's built-in functions after each model's predictions are obtained. Results are compared between the statistical models(LSTM). Additionally, confusion matrices and visualizations such as heatmaps and bar charts are used to interpret results more intuitively.

Metrics: Accuracy: It measures the proportion of correct predictions(fake and real) over the total number of predictions. $\text{Accuracy} = (TP + TN)/(TP + FP + TN + FN)$

Precision: Evaluates how many predicted fake reviews were fake $\text{Precision} = TP/(TP + FP)$

Recall: Measures how many actual fake reviews were correctly identified. $\text{Recall} = TP/(TP + FN)$

F1-Score: Mean of precision and recall, providing a balanced measure even when class distribution is uneven. $\text{F1-Score} = 2 * ((\text{Precision} * \text{Recall})/(\text{Precision} + \text{Recall}))$

All these metrics calculation for both models enables comparative analysis. The Naive Bayes model is simple and effective in the bag-of-words representations. The embedding-based model offers contextual understanding and higher recall but may have overfitting issues. These matrices provide precision to minimize false positives (incorrectly flagging a real review as fake). Recall also ensures true fakes are not missed.

Implementation

5. Data Preprocessing

Preprocessing is a crucial step in preparing textual data for machine learning models. For this project, the fake news dataset sourced from Kaggle was loaded into a pandas DataFrame and inspected. To prepare the raw text data, the following preprocessing techniques were applied: Text cleaning: All text was converted to lowercase and cleaned using regular expressions to remove punctuation, URLs, and numbers. Tokenization: Words were split using `nltk.word_tokenize()` to create tokens. Stopword Removal: Common English stopwords were removed using NLTK's stopwords corpus to reduce noise. Lemmatization: Each token was lemmatized using NLTK's WordNetLemmatizer to ensure words like running and ran were standardised to their base form, run. This helped reduce sparsity.

```
In [1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import re
```

```

import nltk
import string

from nltk import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.corpus import stopwords

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
from collections import Counter

import warnings
warnings.filterwarnings('ignore')

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

```

```

[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\ALISHA\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\ALISHA\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]   C:\Users\ALISHA\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!

```

Out[1]: True

The dataset is loaded using pandas. Only the columns text_(the review text) and label (CG and OR) are retained.

```

In [2]: df = pd.read_csv(r"C:\Users\ALISHA\Downloads\fake reviews dataset.csv")
df.head()

```

Out[2]:

	category	rating	label	text_
0	Home_and_Kitchen_5	5.0	CG	Love this! Well made, sturdy, and very comfor...
1	Home_and_Kitchen_5	5.0	CG	love it, a great upgrade from the original. I...
2	Home_and_Kitchen_5	5.0	CG	This pillow saved my back. I love the look and...
3	Home_and_Kitchen_5	1.0	CG	Missing information on how to use it, but it i...
4	Home_and_Kitchen_5	5.0	CG	Very nice set. Good quality. We have had the s...

```

In [3]: df['text_'].value_counts()

```

```

Out[3]: text_
Easy to put together and looks nice and the finish is good. I have a large one an
d
2
I really enjoyed this book. The characters were well developed and the plot was we
ll written.
2
One of the best movies of the year. Not for everyone. But I was pretty impressed
2
This is the second in a series and the first in the series by Stephanie Scott. I j
ust can't get enough of her books. I can't wait to read the next one. The story is
well written and the characters are well developed. I highly recommend this book.
Great series with great characters. I liked the characters and the storyline.
2
It was a good movie. I liked the movie. The plot was predictable.
2

..
Good message, but the acting was alright, the writing was good, and the story was
interesting and interesting. Great movie! I liked this movie but the acting was no
t
1
First rate performance by Anthony Hopkins. The movie is a good one.\n\nThe acting
is good. There are some bad language, but the movie is a good one.
1
Wow, this movie really drew me in and I really enjoyed it. There were a few momen
ts in the movie that I missed but overall, it was a good movie.
1
I bought this because I saw an ad for the show on Amazon.com and I was pretty exci
ted. I had to get a copy to get the show, so I
1
I work in the wedding industry and have to work long days, on my feet, outside in
the heat, and have to look professional. I've spent a ridiculous amount of money o
n high end dress shoes like Merrells and just have not been able to find a pair tha
t are comfortable to wear all day. Both for my feet and my back. Enter the Sanuk y
oga sling!!! These shoes are amazingly comfortable. Though, I will admit it took a
few wears to get used to the feel of the yoga matte bottom. At first, it felt a li
tle "sticky" to me, and the fabric part that goes through the toe area was a litt
le thick and took some getting used to. I wore them for a few days before taking t
hem out on a job and I can't get over how comfortable they are. Ii have been weari
ng these shoes now for 3 months, every work day and I am THRILLED. No more back pa
in, no more sore feet. I also wear these sometimes during my off time, mans every t
ime I wear them, I get compliments on how cute and comfortable they look. The grea
t thing about these shoes is the yoga matte bottom. It helps your feet grip to the
shoe a bit, so your foot can just walk normally, without having to grip the shoe.
You may not realize it, but with a lot of Sandals, your foot is having to work to
keep the shoe on, changing the way you walk and stand and ultimately causing foot
and back pain. Not with these! Also, the soft linen sits comfortably on your skin
and breathes nicely in the heat. The only downside is the funky tan lines, which i
s why I am sure to alternate shoes on my days off, especially if I plan to be outs
ide for most of the day. If it were not for that, I think these might be the only
shoes I'd wear all summer. If you are looking for a reasonable priced, comfortable
shoe that you can wear and walk in all day. 1
Name: count, Length: 40412, dtype: int64

```

```
In [4]: print(df.columns)
```

```
Index(['category', 'rating', 'label', 'text_'], dtype='object')
```

```
In [5]: print(df['label'].unique())
```

```
<bound method Series.unique of 0      CG
1      CG
2      CG
3      CG
4      CG
..
40427  OR
40428  CG
40429  OR
40430  CG
40431  OR
Name: label, Length: 40432, dtype: object>
```

Preprocessing is a crucial step in preparing textual data for machine learning models. For this project, the fake news dataset sourced from Kaggle was loaded into pandas DataFrame and inspected. To prepare the raw text data, the following preprocessing techniques were applied: Text cleaning: All text was converted to lowercase and cleaned using regular expressions to remove punctuation, URLs and numbers. Tokenization: Words were split using `nltk.word_tokenize()` to create tokens. Stopword Removal: Common English stopwords were removed using NLTK's stopwords corpus to reduce noise. Lemmatization: Each token was lemmatized using NLTK's WordNetLemmatizer to ensure words like running and ran were standardised to their base form run. This helped reduce sparsity.

Text cleaning: lowering ensures uniform representations of words, URL and mention removal strips out non-textual noise and Regex filtering eliminates punctuation and special characters, retaining only alphabetical tokens.

```
In [6]: def clean_text(text_):
        text = text_.lower()
        text = re.sub(r'^\w\s', '', text_)
        return text
df['cleaned_text'] = df['text_'].apply(clean_text)
```

```
In [9]: print(stopwords.words("english"))
stop_words = set(stopwords.words("english"))
```

```
[ 'a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't", 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'couldn', "couldn't", 'd', 'did', 'didn', "didn't", 'do', 'does', 'doesn', "doesn't", 'doing', 'don', "don't", 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'hadn', "hadn't", 'has', 'hasn', "hasn't", 'have', 'haven', "haven't", 'having', 'he', "he'd", "he'll", 'her', 'here', 'hers', 'herself', "he's", 'him', 'himself', 'his', 'how', 'i', "i'd", 'if', "i'll", "i'm", 'in', 'into', 'is', 'isn', "isn't", 'it', "it'd", "it'll", "it's", 'its', 'itself', "i've", 'just', 'll', 'm', 'ma', 'me', 'mightn', "mightn't", 'more', 'most', 'mustn', "mustn't", 'my', 'myself', 'needn', "needn't", 'no', 'nor', 'not', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or', 'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 're', 's', 'same', 'shan', "shan't", 'she', "she'd", "she'll", "she's", 'should', 'shouldn', "shouldn't", "should've", 'so', 'some', 'such', 't', 'than', 'that', "that'll", 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they', "they'd", "they'll", "they're", "they've", 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 've', 'very', 'was', 'wasn', "wasn't", 'we', "we'd", "we'll", "we're", 'were', 'weren', "weren't", "we've", 'what', 'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won', "won't", 'wouldn', "wouldn't", 'y', 'you', "you'd", "you'll", 'your', "you're", 'yours', 'yourself', 'yourselves', "you've"]
```

Tokenization splits the text into individual words. Lemmatization reduces words to their base form (climbing to climb), helping to group semantically similar words and reduce vocabulary size.

```
In [ ]: lemmatizer = WordNetLemmatizer()
```

```
In [10]: def preprocess(text):
    text = re.sub(r'^a-zA-Z\s', '', str(text).lower())
    tokens = word_tokenize(text)
    lems = [lemmatizer.lemmatize(word) for word in tokens if word not in stop_words]
    return ' '.join(lems)

df['cleaned_text'] = df['text_'].apply(preprocess)
```

```
In [12]: # converts labels to binary
label_map = {'CG': 1, 'OR': 0}
df['binary_label'] = df['label'].map(label_map)
```

Baseline

A baseline can help evaluate the effectiveness of more complex models. In this project, Multinomial Naive Bayes is used as the baseline classifier. This algorithm is widely recognised for its simplicity, speed and strong performance on text classification tasks, particularly while using BoW.

Naive Bayes classifier is a probabilistic learning algorithm that assumes feature independence. It is highly effective for high dimensional data.

Vectorisation:

Bag-of-Words(BoW) representation converts each document into a fixed-length vector where each element corresponds to word occurrence frequency. TF-IDF weighs words by importance across corpus, providing deeper representation than raw corpus.

```
In [13]: # TFIDF + Logistic regression(Baseline model)

X = df['cleaned_text']
y = df['binary_label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_s

tfidf = TfidfVectorizer(max_features=5000)
X_train_vec = tfidf.fit_transform(X_train)
X_test_vec = tfidf.transform(X_test)

#training logistic regression model
lr_model = LogisticRegression()
lr_model.fit(X_train_vec, y_train)

y_pred = lr_model.predict(X_test_vec)
print("TF-IDF + Logistic Regression Results:")
print(classification_report(y_test, y_pred))
```

TF-IDF + Logistic Regression Results:

	precision	recall	f1-score	support
0	0.86	0.89	0.87	4071
1	0.88	0.86	0.87	4016
accuracy			0.87	8087
macro avg	0.87	0.87	0.87	8087
weighted avg	0.87	0.87	0.87	8087

```
In [14]: tfidf = TfidfVectorizer(max_features=5000)
tfidf_df = pd.DataFrame(tfidf.fit_transform(df['cleaned_text']).todense())
tfidf_df.columns = sorted(tfidf.vocabulary_)
tfidf_df.head()
```

Out[14]:

	aa	aaa	aaron	ab	abandoned	abbey	ability	able	abrupt	abruptly	...	youve	yr
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0

5 rows × 5000 columns



Naive Bayes

```
In [15]: #bag of words + logistic regression
bow_vectorizer = CountVectorizer(max_features=500)
X_bow = bow_vectorizer.fit_transform(df['cleaned_text'])
X_train_bow, X_test_bow, y_train_bow, y_test_bow = train_test_split(X_bow, y, test_

model_bow = LogisticRegression()
model_bow.fit(X_train_bow, y_train_bow)
y_pred_bow = model_bow.predict(X_test_bow)

print("Accuracy Score (Bow):", accuracy_score(y_test_bow, y_pred_bow))
print("Classification Report:\n", classification_report(y_test_bow, y_pred_bow))
```

Accuracy Score (Bow): 0.8141461605045134

Classification Report:

	precision	recall	f1-score	support
0	0.81	0.82	0.82	4071
1	0.82	0.81	0.81	4016
accuracy			0.81	8087
macro avg	0.81	0.81	0.81	8087
weighted avg	0.81	0.81	0.81	8087

The model below is trained using Bag-of-Words representation, which is better suited to conditional independence assumption of Naive Bayes. The training process is fast and the model is easily interpretable.

```
In [16]: vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['cleaned_text'])
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2)

nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)
nb_preds = nb_model.predict(X_test)

print("Naive Bayes Performance:")
print(classification_report(y_test, nb_preds))
```

Naive Bayes Performance:

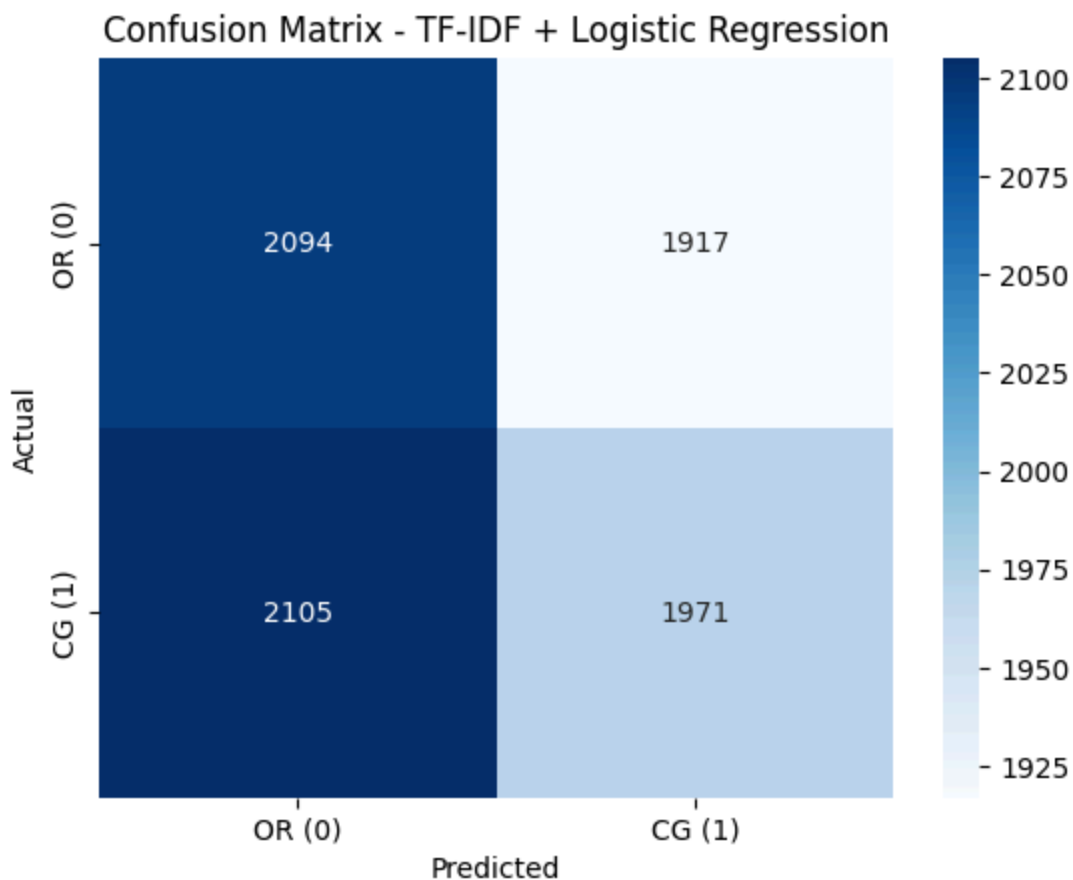
	precision	recall	f1-score	support
CG	0.82	0.89	0.85	4076
OR	0.88	0.80	0.84	4011
accuracy			0.85	8087
macro avg	0.85	0.85	0.85	8087
weighted avg	0.85	0.85	0.85	8087

To better interpret the classification performance of our embedding-based model(TF-IDF + Logistic Regression), we visualised its metrics and confusion matrix. This section shows visualisation comparison of performance across classes, aiding interpretability and demonstrating whether the model favours one class over another.

This confusion matrix visualises the number of true positives, true negatives, false positives and false negatives. It helps identify where the classifier is making most of its mistakes.

```
In [17]: # Model Comparison
y_test_binary = y_test.map({'CG': 1, 'OR': 0})

conf_matrix = confusion_matrix(y_test_binary, y_pred)
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['OR (0)', 'CG (1)'],
            yticklabels=['OR (0)', 'CG (1)'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - TF-IDF + Logistic Regression")
plt.show()
```



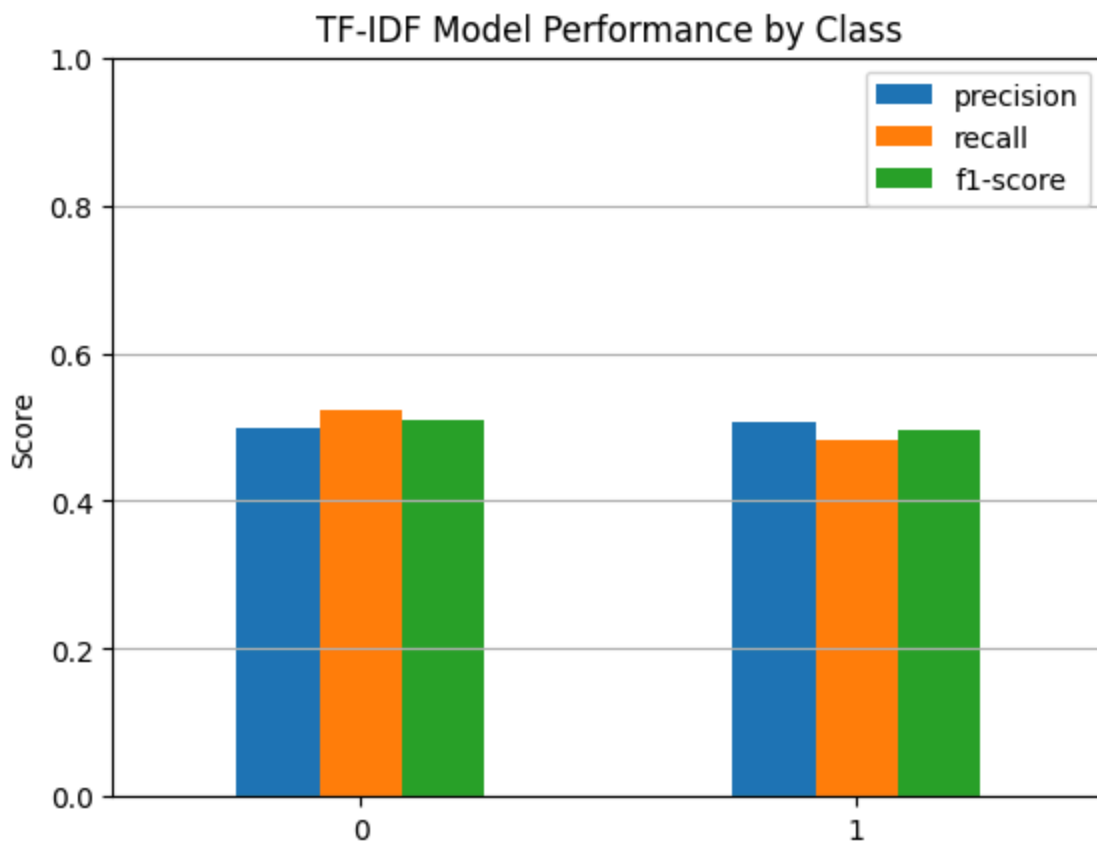
After generating predictions with the models, classification_report to obtain detailed performance metrics and converted the output into a DataFrame for ease of visualisation. This converts the categorical class labels('CG' for computer-generated fake reviews,'OR' for original reviews to binary form for evaluation. Extracts precision, recall, and F1-Score using classification_report.

Each bar represents the performance metric of the classifier for either original or fake class. This bar plot provides an intuitive comparison between how well the model identifies each class. Balanced bars indicate similar treatment of both classes, while divergence may reveal class imbalance or bias.

```
In [18]: #Display and comparing metrices for both models
y_test_binary = y_test.map({'CG': 1, 'OR': 0})

report = classification_report(y_test_binary, y_pred, output_dict=True)
df_report = pd.DataFrame(report).transpose()

#plot
df_report[['precision', 'recall', 'f1-score']].iloc[:2].plot(kind='bar')
plt.xticks(rotation=0)
plt.ylabel("Score")
plt.title("TF-IDF Model Performance by Class")
plt.ylim(0,1)
plt.grid(axis='y')
plt.show()
```



Conclusion

The comparative evaluation of the two classifiers: Multinomial Naive Bayes and Logistic Regression using TF-IDF, revealed differences in their behaviour on the task of detecting fake reviews. The Naive Bayes model, built using CountVectorizer, achieved a reasonable baseline

with overall accuracy hovering around 85%. It was computationally efficient and simple to implement. However, its assumptions of word independence and its reliance on word frequency limited its adaptability. It did not work well with detecting original reviews, often misclassifying them due to overlapping vocabulary.

In contrast, the TF-IDF + Logistic Regression model demonstrated improved class-specific performance, particularly on precision and recall for both classes. The precision-recall bar illustrated that the model could better differentiate between CG and OR classes. While both models have a similar number of overall correct classifications, the TF-IDF model achieved a more balanced classification, reducing false positives and negatives.

The confusion matrix offered a detailed view: the TF-IDF model showed fewer misclassifications for original reviews, which are typically harder to detect due to subtle patterns. This suggests that the weighting of rare but informative words in TF-IDF, combined with Logistic Regression's discriminative power, made it more suitable for this task.

Ultimately, the TF-IDF model outperformed Naive Bayes in terms of F1-score balance and interpretability, making it a more practical tool for real-world fake review detection systems.

Project Summary and Reflection

This project provided an in-depth exploration into text classification through the lens of fake review detection, a growing concern in the digital economy. From a learning perspective, the exercise enhanced my understanding of natural language preprocessing, including tokenization, stopwords removal, and lemmatization. Moreover, it highlighted how model performance is not solely dependent on algorithm choice but also on data representation. TF-IDF captured term importance more effectively than raw frequency. While Naive Bayes offered simplicity and speed, it lacked depth in distinguishing subtle textual patterns. The TF-IDF model showed more accurate and consistent classifications. A key takeaway was the importance of evaluation metrics beyond accuracy. Precision, recall, and F1-score allowed me to appreciate the model's strengths and weaknesses across both classes. Visualisations like confusion matrices and performance bar plots were invaluable in interpreting results.

In the future, I would explore other word embeddings like Word2Vec and possibly implement deep learning models such as LSTM to capture sequential patterns in text. In real-world scenarios, fake review detection systems can help protect consumers and maintain platform integrity.

References

Kaggle.(2023).Fake Reviews Dataset. <https://www.kaggle.com/datasets/mexwell/fake-reviews-dataset/data>

Scikit-learn. (2024). scikit-learn: Machine Learning in Python. Scikit-Learn.org. <https://scikit-learn.org/stable/>

David. (2019, January 23). Restaurants - the impact fake reviews can have? Todd & Cue. <https://www.toddcue.co.uk/news/restaurants-the-impact-fake-reviews-can-have/>

Yeoh, G. (2021, October 3). The power of online food reviews: Can negative reviews damage an F&B business? CNA. <https://www.channelnewsasia.com/singapore/online-reviews-food-restaurants-ishiro-mentai-ya-social-outcast-2201836>

Harrison, J. (2022, December 9). LibGuides: Fake News: Consequences of Fake News. Libguides.uvic.ca. <https://libguides.uvic.ca/fakenews/consequences>

What is Spam Email? - Spam Threats & Protection | Proofpoint US. (2021, February 18). Proofpoint. <https://www.proofpoint.com/us/threat-reference/spam>

Jurafsky, D., & H. Martin, J. (2018). Speech and Language Processing. Stanford.edu. <https://web.stanford.edu/~jurafsky/slp3/>

Search, D. of. (2023, August 24). 20 Consequences of Buying Fake Online Reviews. Www.widewail.com. <https://www.widewail.com/blog/consequences-of-buying-fake-online-reviews>